

THE REAL-WORLD STORY

Imagine a treasure hunt where each clue box contains a gold coin and a paper slip giving the exact GPS location of the next clue box.

THE LINKED LIST AHA MOMENT

Unlike arrays requiring contiguous memory blocks, linked list nodes live anywhere in RAM! Inserting or deleting a node at the head or middle takes **O(1) pointer updates** without shifting elements!

WHY ARRAYS VS LINKED LISTS?

- Array Insert at Index 0:** Slow $O(N)$ shift of all elements.
- Linked List Insert at Head:** Fast $O(1)$ pointer updates: ``newNode.next = head; head = newNode``.
- Random Access:** Array is $O(1)$ via ``arr[i]``, Linked List is $O(N)$ via pointer traversal.

VISUALIZING 3-POINTER REVERSAL

Step-by-step Pointer Reversal Mechanics:

```
Initial: prev=null curr -> [1]
-> [2] -> [3] -> null
              next = curr.
next ([2])
```

```
Step 1: curr.next = prev (point
s [1] back to null)
Step 2: prev = curr      (moves
prev to [1])
Step 3: curr = next      (moves
curr to [2])
```

```
Result: null <- [1] <- [2] <-
[3] (prev is new head!)
```

PREREQUISITES & COMPLEXITY REASONING

Operation / Pattern	Time Complexity	Auxiliary Space
Reverse Linked List (3 Pointers)	O(N) single pass	O(1) in-place
Detect Cycle (Floyd's Fast & Slow)	O(N) single pass	O(1) space
Merge Two Sorted Lists (Dummy Head)	O(N + M)	O(1) space
Reorder List (Find Mid + Reverse + Merge)	O(N)	O(1) space

CLUES THAT SCREAM LINKED LIST

- "Reverse in-place in $O(1)$ space" -> 3-Pointer Iterative Reversal (``prev``, ``curr``, ``next``)
- "Find middle / Check for cycle" -> Fast & Slow Pointers (``slow = slow.next``, ``fast = fast.next.next``)
- "Modify head pointer dynamically" -> Dummy Head Node Sentinel (``ListNode dummy = new ListNode(0)``)

PATTERN 1

REVERSE LINKED LIST (3-Pointer Iterative Reversal)

e.g. Reverse Linked List (LC 206)

WHEN TO USE

Reverse direction of all next pointers in-place in $O(N)$ time and $O(1)$ space.

TEMPLATE — LC 206

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

PATTERN 2 MERGE TWO SORTED LISTS (Dummy Sentinel Node) e.g. Merge Two Sorted Lists (LC 21)

WHEN TO USE

Combine two pre-sorted lists into a single sorted list using a dummy head node in $O(N + M)$ time.

TEMPLATE — LC 21

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
        else { curr.next = l2; l2 = l2.next; }
        curr = curr.next;
    }
    curr.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

PATTERN
5

REMOVE NTH NODE FROM END (Fast Ahead Gap Window)

e.g. Remove Nth Node From End of List (LC 19)

N-STEP GAP WINDOW

Advance `fast` N steps ahead. Then move `fast` and `slow` together until `fast.next == null`. `slow` will point directly before target node!

TEMPLATE — LC 19

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0, head);
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next; // Bypass target node!
    return dummy.next;
}
```

PATTERN 6

ADD TWO NUMBERS (Digit Carry Propagation) e.g. Add Two Numbers (LC 2)

CARRY PROPAGATION

Traverse both lists digit by digit. Sum ``val1 + val2 + carry``. New digit node = ``sum % 10``, ``carry = sum / 10``.

TEMPLATE — LC 2

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int v1 = (l1 != null) ? l1.val : 0;
        int v2 = (l2 != null) ? l2.val : 0;
        int sum = v1 + v2 + carry;
        carry = sum / 10;
        curr.next = new ListNode(sum % 10);
        curr = curr.next;
        if (l1 != null) l1 = l1.next;
        if (l2 != null) l2 = l2.next;
    }
    return dummy.next;
}
```

COMPLETE LINKED LIST PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
206	Reverse Linked List	3-Pointer Iterative Reversal	O(N)	O(1)	E
21	Merge Two Sorted Lists	Dummy Node Sentinel	O(N + M)	O(1)	E
141	Linked List Cycle	Floyd's Fast & Slow Pointers	O(N)	O(1)	E
143	Reorder List	Mid + Reverse + Interleave	O(N)	O(1)	M
19	Remove Nth Node From End	Fast Ahead Gap Window	O(N)	O(1)	M
2	Add Two Numbers	Digit Carry Propagation	O(max(N, M))	O(1)	M
25	Reverse Nodes in k-Group	Group Reversal Loop	O(N)	O(1)	H

LC 206 · Reverse Linked List

EASY

Pattern: 3-Pointer
Iterative

Key Insight:
`curr.next = prev`.

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode nxt = curr.next; curr.next = prev; prev = curr; curr = nxt;
    }
    return prev;
}
```

LC 21 · Merge Two Sorted Lists

EASY

Pattern: Dummy
Node Sentinel
Key Insight:
Compare `l1.val` vs `l2.val`.

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), c = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { c.next = l1; l1 = l1.next; }
        else { c.next = l2; l2 = l2.next; }
        c = c.next;
    }
    c.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

LC 141 · Linked List Cycle

EASY

Pattern: Fast & Slow
Pointers

Key Insight: Check
`slow == fast`.

```
public boolean hasCycle(ListNode head) {
    ListNode s = head, f = head;
    while (f != null && f.next != null) {
        s = s.next; f = f.next.next;
        if (s == f) return true;
    }
    return false;
}
```


LC 143 · Reorder List

MEDIUM

Pattern: Mid + Reverse + Interleave

Key Insight: Split at mid, reverse right, merge.

```
public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;
    ListNode s = head, f = head;
    while (f != null && f.next != null) { s = s.next; f = f.next.next; }
    ListNode prev = null, curr = s.next; s.next = null;
    while (curr != null) { ListNode n = curr.next; curr.next = prev; prev = curr; curr = n; }
    ListNode fst = head, snd = prev;
    while (snd != null) {
        ListNode t1 = fst.next, t2 = snd.next;
        fst.next = snd; snd.next = t1;
        fst = t1; snd = t2;
    }
}
```

LC 19 · Remove Nth Node From End

MEDIUM

Pattern: Fast Ahead Gap Window

Key Insight: Advance `fast` N steps ahead.

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0, head);
    ListNode f = dummy, s = dummy;
    for (int i = 0; i < n; i++) f = f.next;
    while (f.next != null) { f = f.next; s = s.next; }
    s.next = s.next.next;
    return dummy.next;
}
```

LC 25 · Reverse Nodes in k-Group

HARD

Pattern: Group Reversal Loop

Key Insight: Check `getKth()`, reverse k nodes.

```
public ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0, head), gPrev = dummy;
    while (true) {
        ListNode kth = getKth(gPrev, k);
        if (kth == null) break;
        ListNode gNxt = kth.next, prev = kth.next, curr = gPrev.next;
        while (curr != gNxt) { ListNode t = curr.next; curr.next = prev; prev = curr; curr = t; }
        ListNode tmp = gPrev.next; gPrev.next = kth; gPrev = tmp;
    }
    return dummy.next;
}
private ListNode getKth(ListNode c, int k) { while (c != null && k > 0) { c = c.next; k--; } return c; }
```

LC 2 · Add Two Numbers

MEDIUM

Pattern: Digit Carry Propagation

Key Insight: `val1 + val2 + carry`.

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), c = dummy; int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int v1 = (l1 != null) ? l1.val : 0, v2 = (l2 != null) ? l2.val : 0;
        int sum = v1 + v2 + carry; carry = sum / 10;
        c.next = new ListNode(sum % 10); c = c.next;
        if (l1 != null) l1 = l1.next; if (l2 != null) l2 = l2.next;
    }
    return dummy.next;
}
```

🔍 DRY RUN — 3-Pointer Reversal ([1] -> [2] -> [3] -> null)

Step	prev	curr	nextTemp	curr.next set to	State
1	null	1	2	null ('1.next = null')	null <- [1]
2	1	2	3	1 ('2.next = 1')	null <- [1] <- [2]
3	2	3	null	2 ('3.next = 2')	null <- [1] <- [2] <- [3] ✓

📐 MATHEMATICAL PROOF — FLOYD'S CYCLE DETECTION O(N)

Claim: Fast (2x speed) and Slow (1x speed) pointers are guaranteed to meet inside a cycle of length C.

Proof: At each step, distance between fast and slow decreases by 1 ('2 - 1 = 1'). Inside a cycle of length C, fast reduces gap from C to 0 in at most C steps. Total steps <= N -> **Exact O(N) Linear Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Code 3-pointer list reversal iteratively in 2 minutes
- ☐ Implement Merge Two Sorted Lists using Dummy Head
- ☐ Detect cycle using Floyd's Fast & Slow Pointers
- ☐ Find middle node using Fast & Slow Pointers
- ☐ Code Reorder List (Mid + Reverse + Interleave)
- ☐ Remove N-th node from end using N-step gap
- ☐ Code Reverse Nodes in k-Group in O(1) space
- ☐ Prove why Floyd's cycle detection runs in O(N)

📦 GOLDEN RULES — NEVER FORGET

- Rule 1 — Always Save Next Pointer Before Reconnecting**

In pointer reversal, ALWAYS store 'ListNode nextTemp = curr.next' before executing 'curr.next = prev'! Otherwise, you lose reference to the rest of the list!
- Rule 2 — Use Dummy Node for Head Modifications**

Whenever deleting or inserting at the head of a linked list, initialize 'ListNode dummy = new ListNode(0, head)' to avoid special null check branching!